Suborn: A Threat Model for LLM-Orchestrated Firmware Agents

On the insertion of autonomous language model controllers
into the pre-OS execution environment

Ivan Gaydardzhiev
Independent Security Researcher

2026

Abstract

This paper defines a threat class we call the *LLM-orchestrated firmware agent*: a persistent implant embedded in the pre-OS firmware execution environment whose command selection is delegated to a remote large language model rather than encoded in the implant itself. We describe two insertion substrates, the Coreboot ramstage and the Intel Management Engine, analyze the properties of each, identify the detection gaps this architecture creates relative to conventional firmware implants, enumerate the open problems it poses for defenders, and characterize the conditions under which a locally resident inference model might eventually replace the remote LLM dependency. We further analyze the agent's capacity for LLM-directed self-modification: because the ramstage executes with direct access to the SPI flash containing the firmware image, the LLM controller can instruct the agent to rewrite its own binary in response to detection signals, rotate its network endpoints, extend its primitive table with new code, or alter legitimate firmware components outside its own execution context. This collapses the one asymmetry previously available to defenders, that the firmware image is a complete and stable inventory of the agent's capabilities, and introduces a class of adaptive evasion with no adequate forensic response in the current literature.

Contents

Introduction

Firmware-level implants are not a new category. The research literature has documented them continuously since at least 2006, when Rutkowska demonstrated hypervisor rootkits that placed themselves below the operating system's visibility horizon [2]. Subsequent work by Invisible Things Lab on Intel TXT and SINIT [3], by the coreboot security community on CBFS integrity, and by Eclypsium on BMC and UEFI implants [4] has established that code executing before the OS loader is difficult to detect, survives OS reinstallation and disk replacement, and in some configurations persists across firmware updates.

What this body of work does not address is what happens when the firmware implant's decision engine is not a static command table but a large language model queried over the network. The distinction matters because it changes the detection surface in ways that current firmware defense tooling does not account for.

A conventional firmware implant contains its attack logic. Static analysis of the firmware image can, in principle, recover that logic. The command vocabulary is bounded and enumerable. The trigger conditions are hardcoded. The behavior is fully predictable from the binary.

An LLM-orchestrated agent inverts this. The firmware image contains only a trigger detection loop, a set of hardware primitives, and a network transport. The logic that selects which primitives to invoke, in what order, with what parameters, and in response to what telemetry, resides remotely. The firmware binary itself contains no attack intent in the conventional sense. It contains capability and transport. The intent is generated at runtime.

The distinction is not marginal: it breaks the assumption on which firmware image analysis is based.

Scope

This paper covers:

- The threat model and attack class definition (Section 2)
- Two concrete insertion substrates: Coreboot ramstage and Intel ME (Section 3)
- The LLM controller architecture and its properties, including self-modification (Section 4)
- Detection gaps created by this architecture (Section 5)
- The open question of local inference in ME-class hardware (Section 6)
- Open problems for defenders (Section 7)

This paper does not cover: specific exploitation techniques for any of the insertion vectors described, working primitive implementations, or methods for compromising firmware signing infrastructure. The insertion vectors are identified and characterized at the architectural level; the specific vulnerability chains that realize them in any given platform are outside the paper's scope.

Naming

Suborn: to induce a person to commit a wrongful act through corruption or persuasion. Applied to firmware: to subvert the trusted execution substrate from within, whether by corrupting the build pipeline, compromising the update mechanism, exploiting a firmware vulnerability, or reaching the flash directly. The name refers to the act of suborning the substrate, not to the method.

Threat Model

Attacker position

The threat class described in this paper is insertion-agnostic. What is required is that the agent binary reaches the pre-OS execution environment and persists there. The insertion vector determines the attacker's required capability and the detection surface of the insertion event itself, but it does not change any property of the agent once resident: its behavior, its detection gaps, and its forensic properties are identical regardless of how it arrived.

The principal insertion vectors are as follows.

Build pipeline compromise. Access to the coreboot source tree and build system is sufficient to insert a module that will be compiled and linked into the firmware image. The resulting binary is signed along with the rest of the firmware if the platform uses firmware signing, and the signature covers the tampered image. This is the lowest-privilege insertion path: no physical access to the target device is required, and the compromise is inserted before the device is shipped. Supply chain compromise at the firmware level has been demonstrated against major OEM build infrastructure.

Physical access via external programmer. An attacker with brief physical access to the target device can remove or clip onto the SPI flash chip and write an arbitrary firmware image directly, bypassing all software-enforced signing and access controls. This is the classic evil maid scenario applied to firmware. It requires no vulnerability in the running system and defeats signing verification entirely, since the attacker writes the flash directly rather than going through the platform's update mechanism.

OS-level exploitation of firmware interfaces. A vulnerability in a UEFI DXE driver, a SMM handler, a BMC interface, or any other firmware component reachable from the running OS can allow an attacker to write to SPI flash or inject code into the firmware execution environment without physical access. The history of UEFI security research is a continuous record of such vulnerabilities. An agent implanted through this vector is indistinguishable at the firmware level from one implanted through any other means.

Exploitation of firmware update mechanisms. Firmware update paths that do not correctly verify image authenticity or that trust the update payload unconditionally allow an attacker with OS-level access to deliver a tampered image through the platform's own update infrastructure. This vector has been demonstrated repeatedly against BMC and UEFI update mechanisms in production hardware.

The paper uses build pipeline compromise as the primary illustrative case because it represents the minimum attacker capability required for ramstage insertion, not because it is the only or even the most common path. The ME substrate requires higher privilege for any insertion vector, as described in Section 3.2.

Attacker goals

The attacker wants persistent, covert access to the target platform that survives OS reinstallation, that is invisible to host OS security tooling, and that does not require maintaining a persistent network connection to the target. The agent activates on a trigger, performs its tasked operations, and returns to a dormant state.

The LLM controller serves a specific goal within this: the attacker wants the agent's behavior to be adaptive, contextual, and not fully predictable from analysis of the firmware image alone. This reduces the value of static analysis as a detection mechanism.

Attack class definition

An *LLM-orchestrated firmware agent* is a firmware implant with the following properties:

1. It executes in the pre-OS environment, before any operating system code runs.
2. It maintains a set of hardware primitives, functions that interact with hardware directly using firmware-level access.
3. It monitors one or more trigger conditions and activates on trigger.
4. On activation, it sends a structured prompt to a remote LLM endpoint, including platform telemetry and accumulated conversation history.
5. The LLM response selects one or more primitives to invoke.
6. The agent invokes the selected primitives, exfiltrates the results, persists its state, and returns to the trigger poll loop.
7. No attack logic is encoded in the firmware image. The firmware image encodes capability and transport only.

This definition distinguishes the threat class from:

- *Conventional firmware implants*, which encode their command logic statically.
- *Firmware backdoors*, which provide persistent access but do not involve autonomous decision-making.
- *OS-level LLM-assisted attack tools*, which operate above the OS boundary and are visible to host security tooling.

Threat severity

Table 1 summarizes the severity properties of this threat class relative to a conventional firmware implant.

Property	Conventional implant	LLM-orchestrated agent
Static analysis reveals attack logic	Yes	No
Command vocabulary is bounded	Yes	No
Behavior is reproducible from binary	Yes	No
Detection requires behavioral analysis	Sometimes	Always
Adapts to detected defenses	No	Yes
Maintains cross-reboot context	Rarely	Yes
Requires network access	Rarely	Yes (remote variant)

Table 1: Threat class comparison

Insertion Substrates

Coreboot ramstage

Coreboot divides the boot process into discrete stages compiled as separate binaries and embedded in the CBFS (Coreboot Filesystem) on SPI flash. The bootblock runs first from flash, initializes the CPU, and loads the romstage. The romstage initializes DRAM using Cache-As-RAM (CAR) as a temporary stack and heap. The postcar stage tears down CAR and decompresses the ramstage from CBFS into regular DRAM. The ramstage then executes with full x86 privilege and access to initialized DRAM.

The ramstage performs all serious hardware initialization. It walks the static device tree, invoking `chip_operations` and `device_operations` callbacks for every discovered device. It writes

ACPI and SMBIOS tables. It loads the payload (typically a bootloader or OS kernel) and transfers control to it.

Hook mechanism

Coreboot provides a structured callback registration system for the ramstage through the boot-state machine [8]. The machine defines a fixed sequence of named states: `BS_PRE_DEVICE`, `BS_DEV_INIT_CHIPS`, `BS_DEV_ENUMERATE`, `BS_DEV_RESOURCES`, `BS_DEV_ENABLE`, `BS_DEV_INIT`, `BS_POST_DEVICE`, `BS_OS_RESUME_CHECK`, `BS_OS_RESUME`, `BS_WRITE_TABLES`, `BS_PAYLOAD_LOAD`, and `BS_PAYLOAD_BOOT`. A module registers a callback against any of these states using the `BOOT_STATE_INIT_ENTRY` macro, specifying the target state and the function to invoke on entry. The callback executes at the designated point in the boot sequence alongside any other registered callbacks for that state.

A mainboard module can also override `mainboard_ramstage_entry` entirely, transferring control permanently to the agent before any device initialization occurs. This is the approach used in `suborn`: the agent installs its own stack and enters a trigger poll loop that never returns.

Insertion point

The insertion point is the coreboot build system. Adding a `Makefile.mk` entry in a mainboard or chipset directory is sufficient to compile and link an additional object file into the ramstage binary. The CBFS packaging tools then embed the resulting ramstage in the flash image. If the platform uses firmware signing, the signature covers the complete image including the injected module.

The injected module is indistinguishable from a legitimate mainboard module at the binary level. Detecting the injection requires either diffing the firmware image against a known-good build or auditing the build system for unauthorized additions.

Capabilities available in ramstage

At the point `mainboard_ramstage_entry` is called, the following are available:

- Initialized DRAM with arbitrary read/write access
- Full x86 Ring 0 privilege
- Direct hardware access via port I/O and memory-mapped I/O
- Access to all PCI devices, including the network interface, USB host controller, and storage controllers
- Access to SMM (System Management Mode) registration interfaces before the OS locks them
- Access to the SPI flash containing the firmware image itself

The OS has not yet started. No security software is running. No network stack is configured at the OS level. The agent operates in an environment with no defensive infrastructure present.

Limitations

The ramstage executes only during boot. If the platform boots infrequently, the agent's activation window is limited. The agent can hook into the ACPI S3 resume path to activate on wakeup from sleep, which significantly increases activation frequency on laptops and workstations.

The ramstage does not persist between power cycles unless the agent explicitly writes state to NVRAM or a CBFS region. `Suborn` implements NVRAM persistence using the CBFS region interface, incrementing a nonce on each activation to provide replay protection.

Intel Management Engine

The Intel Management Engine (CSME from version 11 onward) is the deeper and more capable substrate. It is an independent microcontroller embedded in the Platform Controller Hub (PCH), present on every x86 platform based on Intel chipsets since 2006.

Architecture

ME versions 11 through 14 use a Quark x86 architecture running a customized MINIX 3 microkernel [5]. Later versions use a different internal architecture but maintain the same external interfaces. The ME has:

- Its own CPU, independent of the host processor
- Internal SRAM ranging from 512KB to 1920KB depending on the SKU [6]
- Its own firmware stored in a protected region of SPI flash, signed by Intel with RSA-3072 in CSME 15.0 and later
- Access to a protected region of host DRAM after initialization
- Direct DMA access to host system memory
- Direct access to the network interface, USB controller, and storage
- Continuous operation as long as the platform has standby power, including when the host CPU is off or in a sleep state

The ME communicates with the host OS through the Host Embedded Controller Interface (HECI), which appears to the host as the MEI driver. This bidirectional channel is present and active during normal OS operation.

Persistence properties

An agent running in ME firmware survives:

- OS reinstallation
- Full disk replacement
- Firmware updates that do not include the ME region (common in OEM update packages)
- Host CPU power-off states

The ME firmware region is write-protected by default through SPI flash protection registers configured during platform initialization. Modifying ME firmware requires either a physical SPI programmer or a vulnerability in the ME firmware itself or the flash protection configuration.

Detection difficulty

Host OS tools cannot observe ME firmware execution directly. The ME's internal SRAM, code, and execution state are not accessible through any host-visible interface under normal operation. ME firmware that opens a network connection does so through its own direct NIC access, producing traffic that does not pass through the host OS network stack and is therefore not visible to host-based network monitoring, firewall rules, or endpoint detection software.

Detection requires one of:

- Out-of-band network monitoring at the switch or upstream router level
- Hardware-level SPI flash analysis
- ME firmware image extraction and analysis using specialized tools
- Behavioral detection based on network traffic patterns observed externally

Insertion difficulty

Inserting code into ME firmware is substantially harder than inserting a coreboot ramstage module. ME firmware is signed by Intel. Modifying it requires either a signing key compromise, a ME firmware vulnerability that allows code execution, or physical SPI flash access with the protection registers disabled.

This raises the attacker capability bar significantly. ME-level implants are within the capability of nation-state actors and sophisticated criminal organizations with access to hardware, but are not trivial supply chain insertions in the same way as a coreboot module addition.

The ramstage and ME substrates therefore represent different points on the attacker capability curve. The ramstage is accessible to a supply chain attacker with build system access. The ME requires higher privilege.

The LLM Controller Architecture

Communication model

The agent sends a structured JSON prompt to a remote LLM API endpoint over HTTPS. The prompt is assembled from platform telemetry accumulated since the last activation and contains five fields: a protocol version identifier, a boot nonce that increments on each activation and serves as a unique boot identifier, the name of the trigger condition that caused the current activation, the list of available hardware primitive names representing the agent's full capability surface, and the serialized conversation history from previous activations stored in NVRAM. The response is a JSON object containing a command identifier from the agent's primitive table. The agent parses the command, dispatches to the corresponding primitive, exfiltrates the result back to the LLM endpoint, and persists the updated conversation history to NVRAM.

Conversation memory

The agent maintains conversation history across activations in NVRAM. The LLM controller receives the full history of previous interactions on each call, allowing it to build a contextual picture of the target over time. This enables behavior that depends on accumulated knowledge: adapting the exfiltration strategy based on what has been collected previously, recognizing environmental changes, or deferring certain actions until specific conditions have been observed across multiple boot cycles.

Trigger model

Three trigger conditions are implemented in `suborn`:

1. **USB trigger**: Detection of a USB device insertion matching a specific VID/PID pair, monitored through direct access to the USB host controller registers.
2. **TCP SYN trigger**: Detection of a SYN packet on a specific port, monitored through direct NIC register access before the host OS network stack is initialized.
3. **Serial trigger**: A magic byte sequence received on COM1, useful for out-of-band activation without network connectivity.

Multiple trigger types allow the attacker to activate the agent through different channels depending on physical access, network access, or both.

Killswitch

The agent reads a single byte from a NVRAM location at startup. If the byte is zero, the agent halts permanently. This provides a remote disable mechanism: writing zero to the killswitch

byte prevents future agent activations without removing the agent from the firmware.

The killswitch mechanism is meaningful only if the agent cannot overwrite its own killswitch byte. This requires hardware-level write protection of the relevant NVRAM or SPI flash region, which cannot be assumed across all platforms.

Command primitive table

The agent maintains a table of hardware primitives identified by integer command codes. The LLM controller selects from this table by name in its JSON response. The primitive table represents the agent's full capability surface as compiled. In the absence of self-modification, adding a primitive requires modifying the firmware image, so the capability surface is bounded at firmware build time even though the selection logic is unbounded.

This is a meaningful asymmetry under that constraint: defenders can in principle enumerate the capability surface from the firmware image even when they cannot predict which capabilities will be invoked or in what sequence. The constraint does not hold if the agent includes SPI write capability in its primitive table, as described in Section 4.6.

Self-modification and firmware rewriting

Section 3.1 establishes that at the point `mainboard_ramstage_entry` is called, the agent has direct access to the SPI flash containing the firmware image itself. This is a capability of the substrate, not a capability that needs to be engineered. An agent with a write-to-flash primitive in its table can modify its own binary, and an agent with a read-parse-write primitive can modify any other component of the firmware image resident in CBFS.

The implications of this are substantial and require explicit treatment.

Self-modification in response to detection. If the LLM controller receives telemetry indicating that a specific primitive has been flagged, that network traffic to a specific endpoint has been observed, or that analysis tooling has been run against the firmware image, it can instruct the agent to rewrite its own code before the next boot. The primitive that triggered detection can be removed or replaced. The network endpoint string can be rotated. The trigger logic can be altered. The binary the defender extracts and analyzes after an incident is not necessarily the binary that was executing during the incident, and may not contain the code paths that were active at the time of detection.

Unbounded capability surface. An agent that can write to SPI flash is not limited to the primitives compiled into its initial image. The LLM controller can deliver new executable code as a payload, instruct the agent to write it into a CBFS region, and register it as a new module for execution on the next boot. The capability surface is therefore not bounded at firmware build time in any meaningful sense once SPI write access is available. The one asymmetry previously available to defenders, that the firmware image is a complete inventory of what the agent can do, is eliminated.

Modification of legitimate firmware components. The agent is not limited to modifying its own code. It has access to every CBFS region in the firmware image, including legitimate coreboot modules, ACPI tables, and SMM handler code. An LLM controller with knowledge of the target platform's firmware layout can instruct the agent to modify these components in ways that serve its objectives independently of the agent's own execution, for example by altering SMM handlers to persist across OS reinstallation at a layer below the agent itself, or by modifying ACPI tables to affect OS behavior in ways not attributable to any firmware implant.

Implications for forensic analysis. A self-modifying agent that rewrites its own code after each activation, or after a detected analysis event, presents a forensic record that may be inter-

nally inconsistent. Network traffic observed during one boot cycle corresponds to a binary that may no longer exist on the flash. The NVRAM conversation history, if not also modified, may reference commands that are no longer present in the primitive table. Forensic reconstruction of what the agent did requires not just the current firmware image but a complete history of every version of the image that existed during the agent's operational lifetime, a record that cannot be assumed to exist.

The self-modification capability is not a property unique to LLM-orchestrated agents. Conventional firmware implants can also write to flash. What is new here is that the decision of when to self-modify, what to modify, and how to modify it is delegated to an external controller with contextual knowledge of the target environment and the ability to reason about detection signals. Self-modification becomes adaptive rather than scripted.

- The presence of a trigger poll loop
- The presence of a network transport (HTTPS client code, LLM API endpoint strings)
- The primitive table (command names and dispatch code)

Static analysis will not reveal:

- Which primitives will be invoked
- Under what conditions they will be invoked
- What the agent's accumulated knowledge of the target looks like
- What the LLM controller's strategy is or will be

A firmware image containing a generic HTTP client and a dispatch table is not inherently suspicious. These components have legitimate uses in firmware (network boot, remote management, diagnostics). The attack intent exists only in the LLM controller, which is not present in the firmware image.

Behavioral analysis

Behavioral detection requires observing the agent's network activity. For the ramstage variant, this traffic passes through the host OS network stack after the OS boots, where it is in principle observable. For the ME variant, this traffic bypasses the host OS entirely and is not observable through host tools.

Even for the ramstage variant, the traffic is encrypted HTTPS to a legitimate-appearing endpoint (an LLM API provider). Distinguishing this from legitimate application traffic requires deep packet inspection that can identify LLM API calls specifically, which is not a current capability of standard network security tooling.

Forensic reconstruction

Following an incident, forensic reconstruction of what the agent did requires recovering:

- The firmware image (requiring SPI flash extraction or firmware update package recovery)
- The NVRAM state containing conversation history (may be overwritten or encrypted)
- Network logs from the LLM API provider (requires cooperation with the provider)
- The LLM controller's conversation history and model state (generally inaccessible)

A conventional command-and-control server logs commands issued. An LLM controller maintains state in a model that the defender cannot access. The sequence of decisions the LLM made cannot be reconstructed without that state and the exact model version used at the time.

Signature detection

Signature-based detection of the agent's behavior is not applicable because the behavior is generated dynamically. Command sequences observed during one activation do not predict sequences during subsequent activations. An LLM controller that has been informed (through telemetry) that specific primitives triggered detection can avoid those primitives in future activations.

Local Inference: The ME Substrate Question

The most significant architectural limitation of the design described above is the requirement for network access to reach the LLM controller. This requirement creates an observable network event on every activation and requires the target platform to have internet connectivity.

The natural question is whether a useful inference model could run locally inside the ME, eliminating the network dependency entirely.

Memory constraints

The ME has between 512KB and 1920KB of internal SRAM [6]. It also has access to a protected region of host DRAM after initialization, but this region is bounded and shared with ME's own runtime state.

Current quantized language models require substantially more memory than this. A Qwen 2.5 0.5B model at 4-bit quantization requires approximately 400MB of working memory for inference. The ME SRAM budget is roughly two to three orders of magnitude below this requirement.

The gap is not bridgeable with current model architectures. Even BitNet's 1.58-bit quantization approach [9], which represents the current frontier of weight compression, does not bring a useful general-purpose language model into the sub-2MB range.

Task-specific models

A task-specific model is a different question. A model that does not need to understand natural language, only classify platform telemetry into one of 32 command identifiers, could in principle be trained to be much smaller than a general-purpose language model.

A lookup-table-based classifier or a heavily pruned decision tree trained to map telemetry vectors to command selections could fit in the ME's memory budget. This would provide local decision-making without network access, at the cost of adaptability: the model's decision function is fixed at training time and cannot update based on new information without a firmware update.

The hybrid architecture that seems most plausible in the near term is:

- A small local classifier in ME SRAM for routine trigger responses and first-pass decisions
- A remote LLM call for complex decisions, strategy updates, and situations the local model has not been trained for

This architecture reduces the network call frequency substantially, improving covertness, while retaining the adaptive capability of the remote LLM for cases where it is needed.

Open research question

Whether a useful decision model can fit within 1-2MB of SRAM is an open question with no published answer as of this writing. No research on LLM or ML inference inside ME firmware exists in the public literature. The memory constraints, the Quark x86 instruction set, the

absence of floating-point hardware in early ME versions, and the closed nature of the ME firmware environment all make this a non-trivial research problem. It remains an open problem.

Open Problems for Defenders

The following problems are identified as lacking adequate solutions in the current firmware security literature.

Firmware image auditing at scale. Detecting the insertion of a ramstage module requires diffing every shipped firmware image against a known-good build for that platform. This requires maintaining a database of known-good builds across every platform a defender manages, and a toolchain for automated comparison. This is not standard practice outside of high-security environments.

Pre-OS network monitoring. Traffic generated by a firmware agent before OS boot is not observed by host-based security tools. Capturing it requires network taps at the physical layer or switch-level monitoring. For ME-generated traffic, this is the only available monitoring point regardless of what the OS observes.

LLM API traffic classification. Distinguishing legitimate LLM API calls from agent telemetry requires protocol-aware inspection at a point where the traffic is observable. Current network security tooling does not perform this classification.

Non-deterministic agent behavior. An agent whose behavior is generated by an LLM does not have a fixed behavioral signature. Detection methods based on behavioral patterns observed in previous activations do not generalize to future activations if the LLM controller has access to feedback about what triggered detection.

Forensic recovery of LLM decisions. Reconstructing the sequence of decisions an LLM controller made during an incident requires access to the model, the exact version used, and the full conversation history. The conversation history may be stored in NVRAM (recoverable with hardware access) but the model state at the time of the decisions is generally inaccessible after the fact.

ME firmware integrity verification. Verifying the integrity of ME firmware requires extracting the firmware image and comparing it against Intel's published images. The extraction requires either a physical SPI programmer or a tool that can read the ME region through software (Intel's MEI driver provides limited access; full extraction typically requires hardware). This verification is not performed during routine security assessments.

Killswitch protection. The killswitch mechanism described in Section 4.4 is only meaningful with hardware-enforced write protection of the killswitch storage location. Firmware agents that control their own storage can disable their own killswitch. Defining a killswitch architecture that the agent cannot subvert requires hardware support that is not consistently available across platforms.

Forensic reconstruction against a self-modifying agent. An agent that rewrites its own firmware image in response to detection signals, or on a routine rotation schedule directed by the LLM controller, breaks the assumption that the firmware image extracted after an incident reflects the code that was executing during the incident. There is no existing forensic methodology for reconstructing the history of a self-modifying firmware binary. Each version of the image that existed during the agent's operational lifetime is a distinct artifact, and absent continuous out-of-band firmware monitoring, none of those intermediate versions are recoverable. The problem is not one of tool improvement: the intermediate images were never captured and cannot be reconstructed from what remains.

Proof of Concept

The architecture described in this paper maps directly onto available open tooling. A technically literate reader can construct a validation environment using QEMU as the execution substrate, with QEMU I/O ports standing in for real hardware interfaces and a local HTTP listener standing in for the LLM endpoint. The full agent lifecycle, trigger detection, prompt construction from platform telemetry, command dispatch, and NVRAM state persistence, follows from the architectural description without requiring real hardware or a live LLM connection.

The constraints that such a simulation cannot expose are the same constraints that apply to any emulated firmware environment: real SPI flash timing, hardware-specific initialization sequences, and the precise behavior of ME interfaces under non-standard access patterns. These require validation against real coreboot hardware and are identified as the next step for this research.

Conclusion

The LLM-orchestrated firmware agent is a threat class that does not yet appear in the public threat landscape but follows naturally from the convergence of two existing trends: the maturity of firmware-level implant techniques documented over the past two decades, and the widespread availability of capable LLM APIs accessible over standard HTTPS.

The threat's defining property is the separation of capability from intent. The firmware image encodes what the agent can do. The LLM controller decides what it will do. This separation breaks the assumption underlying firmware image analysis as a detection mechanism and creates a class of agent behavior that cannot be predicted, bounded, or attributed from the binary alone.

The detection gaps identified in Section 5 are not gaps that can be closed by improving existing tools. They require new monitoring infrastructure, new forensic methods, and new assumptions about what constitutes a complete firmware security assessment.

The open question of local inference in ME-class hardware (Section 6) is the direction that would make this threat class significantly harder to detect by removing the network call requirement. It is not solvable with current model architectures but the trajectory of quantization research suggests it should be revisited periodically.

This paper names the threat class, describes its architecture, and identifies what defenders need to address it. The next step is implementation against real coreboot hardware to validate the architectural assumptions and identify the constraints that a simulation cannot expose.

References

[1] Ken Thompson. *Reflections on Trusting Trust.* Communications of the ACM, Vol. 27 No. 8, August 1984.

[2] Joanna Rutkowska. *Subverting Vista Kernel for Fun and Profit.* Black Hat USA, 2006.

[3] Invisible Things Lab. *Attacking Intel Trusted Execution Technology.* Black Hat DC, 2009.

[4] Paul Asadoorian, Eclypsium Research. *Firmware Security Realizations Part 2: Start Your Management Engine.* Eclypsium Blog, August 2022. `https://eclypsium.com/blog/firmware-security-realizations-part-2-start-your-management-engine/`

[5] Wikichip. *Management Engine, Intel.* `https://en.wikichip.org/wiki/intel/management_engine`

[6] Intel Corporation. *Intel Converged Security and Management Engine Security White Paper*. https://www.intel.com/content/dam/www/public/us/en/security-advisory/documents/intel-csme-security-white-paper.pdf

[7] The coreboot Project. *Coreboot Architecture*. https://doc.coreboot.org/getting_started/architecture.html

[8] The coreboot Project. *Ramstage Bootstates and Bootstate Callbacks*. https://doc.coreboot.org/lib/ramstage_bootstates.html

[9] Shuming Ma et al. *BitNet b1.58 2B4T Technical Report*. arXiv:2504.12285, 2025.
